

Prueba Técnica: Frontend Developer (Vue.js + Nuxt + Pinia)

1. Descripción del Proyecto

El candidato deberá desarrollar una aplicación tipo **Dashboard de Gestión de Personajes** utilizando la API pública de **Rick and Morty** (o OpenWeather/PokeAPI). La aplicación debe permitir visualizar datos, gestionar favoritos mediante un estado global y contar con un flujo de acceso restringido.

2. Stack Tecnológico Obligatorio

- **Framework:** Nuxt (Vue - Composition API).
 - **Manejo de Estado:** Pinia.
 - **Estilos:** Tailwind CSS, SCSS o CSS Modules.
 - **Consumo de API:** Fetch, Axios o el uso de `useFetch/useAsyncData` nativos de Nuxt.
-

3. Requerimientos Funcionales

A. Autenticación (Simulada)

- Crear una página de **Login** con un formulario (Email y Contraseña).
- No se requiere un backend real. Si el usuario ingresa cualquier email válido y una contraseña de más de 6 caracteres, debe redirigir al Dashboard.
- **Middleware:** Implementar un middleware de Nuxt que proteja las rutas del Dashboard. Si no hay un "token" (simulado en localStorage/cookie), debe redirigir al Login.

B. Dashboard (Consumo de API)

- Consumir el endpoint de personajes de <https://rickandmortyapi.com/api/character>
- Mostrar una **Grid de Cards** con la imagen, nombre y estado (Alive/Dead) de cada personaje.
- Implementar un **Buscador** que filtre los personajes por nombre.
- **Paginación:** Botones de "Anterior" y "Siguiente".

C. Gestión de Favoritos (Pinia)

- Cada card de personaje debe tener un botón de "Añadir a Favoritos" (ícono de corazón).
 - Crear una sección o vista independiente de **"Mis Favoritos"** donde se listen los personajes seleccionados.
 - El estado debe persistir aunque el usuario navegue entre páginas (usando el store de Pinia).
-

4. Requerimientos Técnicos (Puntos a Evaluar)

1. **Estructura de Componentes:** Separación lógica entre componentes de UI (botones, inputs) y componentes de negocio.
2. **Uso de Composables:** Extraer la lógica de la API a un `composable` reutilizable.
3. **Manejo de Errores:** Mostrar un mensaje o estado de "Loading" mientras se obtienen los datos.
4. **Responsive Design:** La aplicación debe ser funcional en dispositivos móviles y desktop.
5. **Limpieza de Código:** Uso de nombres de variables semánticos y buenas prácticas de TypeScript (opcional pero recomendado).

5. Extras (Valorables)

1. TypeScript básico.
 2. Componentes reutilizables.
 3. Manejo de errores claros.
 4. Tests básicos (Vitest/Jest).
 5. Animaciones/transiciones sutiles.
-

6. Entregables

- **Repositorio de GitHub:** Con un historial de commits claro.
- **Archivo README.md:** Con instrucciones detalladas de cómo instalar y ejecutar el proyecto local.
- **Variables de Entorno** (si aplica).
- **Descripción Breve:** De la arquitectura seleccionada y decisiones relevantes.

7. Criterios de Evaluación

Valoraremos:

A. Correcta implementación de los requisitos:

- Login falso pero funcional.
- Uso de Pinia para auth y dashboard.
- Consumo de API.
- Filtros y paginación.

B. Calidad del código:

- Claridad y organización.
- Uso de componentes.
- Separación de responsabilidades (stores, componentes, helpers).

C. Calidad de la UX/UI:

- Interfaz limpia y entendible.
- Feedback al usuario (loading, errores, no resultados).

D. Extras y atención al detalle:

- TypeScript (si se usa).
- Tests.
- Buen uso de las capacidades de Nuxt (rutas, composables, etc.).